Recursion, Recurrences, and Divide-and-Conquer — Worked solutions

A02-E01 Complete stable MERGE

Complete merge trace

Indices (i, j) before comparison	Selected record	Output prefix
$(1, 1)$	0_a	$[0_a]$
$(2, 1)$	1_d	$[0_a, 1_d]$
$(2, 2)$	Left record 3_b on equality	$[0_a, 1_d, 3_b]$
$(3, 2)$	3_e	$[0_a, 1_d, 3_b, 3_e]$
$(3, 3)$	6_f	$[0_a, 1_d, 3_b, 3_e, 6_f]$

The right side is exhausted; append 7_c without another key comparison. There are 5 key comparisons, 6 copies into independent buffers, and 6 output writes. Loop-bound tests are not counted as key comparisons.

Right priority on equality instead gives $[0_a, 1_d, 3_e, 3_b, 6_f, 7_c]$. It is still an ordered permutation of the input, but is unstable: 3_b originally preceded 3_e. Five comparisons attain the $m - 1$ upper bound for $m = 6$ records.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A02-E02 Sentinels and aliases have assumptions

With sentinels, the buffers are $[M_{\text{real}}, M_{\text{sentinel}}]$ and $[M_{\text{real}}, M_{\text{real}}, M_{\text{sentinel}}]$.

Output step	Read and effect
1	Select the real $L[1]$; advance the left index to 2
2	Select sentinel $L[2]$ on equality; advance to 3
3	The comparison reads nonexistent $L[3]$

A sentinel must rank strictly above every real record; equality with the largest legal key is insufficient.

For the view example, the heads initially hold 2 and 1. Selecting the right 1 writes $A[1] = 1$, overwriting the unread left-head 2. The next comparison sees this overwritten 1, so a record is lost and another is duplicated.

Use independent input copies and explicit length tests. Once one side is exhausted, consume only the other. This needs no out-of-domain value. A sentinel alternative requires a separate tag that ranks above all real records, rather than reusing legal key M.

A02-E03 Odd lengths and peak space

Execution order

(p, q, r)	Merged result	Buffer copies and output writes
$(1, 1, 2)$	$[2_b, 6_a]$	2 each
$(1, 2, 3)$	$[2_b, 5_c, 6_a]$	3 each
$(4, 4, 5)$	$[1_d, 4_e]$	2 each
$(1, 3, 5)$	$[1_d, 2_b, 4_e, 5_c, 6_a]$	5 each

Both cumulative counts are $2 + 3 + 2 + 5 = 12$. The root $(1, 3, 5)$ completes its left call $(1, 2, 3)$, then its right call $(4, 4, 5)$, and finally merges them.

Child buffers have been released when the parent allocates its own. Peak buffer demand is therefore 5 records, distinct from cumulative demand 12. The deepest live call chain has lengths $5 \to 3 \to 2 \to 1$: four frames.

In general, peak record-buffer space is $\Theta(n)$ and the stack uses $\Theta(\lg n)$ fixed-size frames. Cumulative allocation is not simultaneous memory demand.

A02-E04 Exact recurrence solutions

Write $h = \lg n$. Every internal level of U costs $5n$; there are h such levels and n leaves of cost 3. Thus

$$U(n) = 5n \lg n + 3n, \qquad U(8) = 120 + 24 = 144.$$

For W, depth ℓ contributes

$$2^\ell \left(\frac{n}{2^\ell} - 1 \right) = n - 2^\ell.$$

Leaves cost zero, so

$$W(n) = \sum_{\ell=0}^{h-1} (n - 2^\ell) = n \lg n - n + 1, \qquad W(8) = 17.$$

Substitution into each recurrence and verification at $n = 1$ also prove the exact formulas by induction.

Exhaustion depends on the data. Sorted input of length 8 uses 4 key comparisons at each of three internal depths, totaling 12 rather than 17. Buffer copies and writes still each total 8 per internal level; fewer comparisons do not remove all the other work.

A02-E05 Θ bounds beyond powers of two

Balanced floor/ceiling splits place surviving depth-ℓ interval lengths between $\lfloor n/2^\ell \rfloor$ and $\lceil n/2^\ell \rceil$, stopping at singletons. Height is at most $\lceil \lg n \rceil$.

Upper bound. Unfinished intervals at one depth are disjoint with total length at most n. Fixed c_U bounds each internal node's work by c_U times its length. Each level costs at most $c_U n$, giving $O(n \lg n)$.

Lower bound. For $0 \le \ell < \lfloor \lg n \rfloor$, every interval has length at least 2, so all are internal and cover all n records. Standard MERGE copies and writes each interval; fixed $c_L > 0$ therefore gives at least $c_L n$ work per complete level. Since

$$\lfloor \lg n \rfloor \ge \tfrac{1}{2} \lg n \qquad (n \ge 2),$$

these levels supply $\Omega(n \lg n)$.

Exactly n singleton leaves cost $\Theta(n)$ altogether, preserving the main order. Inputs of size 0 and 1 cost $\Theta(1)$ separately. The proof uses actual intervals, not an unproved monotonicity assumption.

A02-E06 Model changes and an early-exit variant

Sorted input requires no insertion shifts, giving $\Theta(n)$ time. Standard MERGE still copies and writes every internal interval, giving $\Theta(n \lg n)$.

After both halves are sorted, $A[q]$ is the left maximum and $A[q+1]$ the right minimum. When $A[q] \le A[q+1]$, concatenation is ordered. Equal left records already precede equal right records, preserving stability when merging is skipped.

Every node skips MERGE on sorted input. The tree has n leaves and $n-1$ internal nodes with constant work per node, so time is $\Theta(n)$. Skipping only removes work, preserving an $O(n \lg n)$ worst-case upper bound. Strictly decreasing distinct keys make every boundary test fail and retain linear merge work at each complete level, proving the matching worst-case lower bound.

Charge the actual cost of each key comparison when it may scan $\Theta(L)$ characters. Treating every comparison as $O(1)$ no longer applies directly. Fixed-size reference moves may still take $O(1)$; multiplying every operation by L is not an exact cost model.

A02-E07 Maximum-subarray states and interval

An interval ending at i either contains A_i alone or extends an interval ending at $i-1$. For extension, the best previous ending sum suffices. Hence

$$E_1 = B_1 = A_1, \quad E_i = \max(A_i, E_{i-1} + A_i), \quad B_i = \max(B_{i-1}, E_i).$$

i	1	2	3	4	5	6	7
A_i	5	-9	2	6	-1	4	-3
E_i	5	-4	2	8	7	11	8
B_i	5	5	5	8	8	11	11

Restart at index 3, then extend through 6: $2 + 6 - 1 + 4 = 11$. An optimal interval is $[3, 6]$. Constant work per iteration gives $\Theta(n)$ time; retaining only current/global endpoints and sums takes $O(1)$ auxiliary space.

All-negative nonempty input selects a largest singleton. Only the empty-permitted version may compare the result with zero and return an empty interval. Initializing B_1 to zero violates the original contract.

A02-E08 Recursion trees and fixed-constant induction

Write $n = 3^L$. Depth j has 3^j nodes of size $n/3^j$, totaling n work. There are L internal levels and n unit-cost leaves, giving

$$T(n) = n(L+1) = n(1+\log_3 n).$$

Let $U(n) = n(1+\log_3 n)$. Then $U(1) = T(1) = 1$ and

$$3U(n/3) + n = n(1+\log_3 n - 1) + n = U(n).$$

Induction proves equality. At $n = 9$, three level costs $9, 9, 9$ total 27.

A guess $T(n) \le cn$ yields only $cn + n$, not the same cn. The induction requires one constant fixed before all input sizes; increasing it to $c+1$ at each level does not close the proof. The correct tight bound is $\Theta(n \lg n)$.

A02-E09 A logarithm-squared boundary

Here $a = 9, b = 3, p = 2$. The ratio $f(n)/n^p = (\lg n)^2$ is unbounded, excluding case 2. It neither tends to zero nor reaches any fixed polynomial n^ϵ, excluding the required gaps in cases 1 and 3.

For $n = 3^L$, depth j costs $n^2(\lg 3)^2(L-j)^2$. Set $r = L - j$. Internal work is

$$n^2(\lg 3)^2 \sum_{r=1}^{L} r^2 = n^2(\lg 3)^2 \frac{L(L+1)(2L+1)}{6} = \Theta(n^2 L^3).$$

Replacing a fixed number of bottom levels by base cases affects only lower-order work. Leaf cost is $\Theta(n^2)$, giving $\Theta(n^2(\lg n)^3)$ overall.

For nine actual children using floor/ceiling division by 3, the chapter's bounded-error proof keeps large actual sizes within fixed factors of $n/3^j$, with only constant depth variation. Away from the base interval, their logarithms are comparable to $L - j$; fixed bottom levels cost $O(n^2)$ separately. The same tight bound follows.

A02-E10 Unequal splits and perturbations

Use the cited continuous theorem with fixed $a_i > 0, 0 < b_i < 1$, positive bounded costs on a fixed base interval, nonnegative differentiable g whose derivative has at most polynomial absolute growth, perturbations $h_i(x) = O(x/\log^2 x)$, and eventual child arguments in $[0, \rho x]$ for fixed $\rho < 1$.

The first recurrence has $a_1 = a_2 = 1$, $b_1 = 1/3, b_2 = 2/3$, $h_i = 0$, and $g(x) = x$, satisfying these conditions. The continuous function

$$F(p) = (1/3)^p + (2/3)^p$$

strictly decreases from infinity to zero as p runs from $-\infty$ to ∞. Thus $F(p) = 1$ has exactly one real root; substitution gives $p = 1$. Therefore

$$T(x) = \Theta\left(x\left[1 + \int_1^x \frac{u}{u^2}\,du\right]\right) = \Theta(x(1 + \ln x)).$$

The second recurrence's first child is $3n/4$. Its additional $n/4$ is not $O(n/\log^2 n)$, so use ratios $3/4, 1/2$. The unique root satisfies

$$(3/4)^p + (1/2)^p = 1, \qquad p \approx 1.507126591639.$$

Since $p > 1$,

$$\int_1^x u^{-p}\,du = \frac{1 - x^{1-p}}{p - 1} = \Theta(1),$$

so the second recurrence is $\Theta(x^p)$. The decimal is approximate; the root equation defines the exact exponent.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A02-E11 Seven products and factor order

Seven products and reconstruction

Product	Substitution	Value
M_1	$(2+4)(5+1)$	36
M_2	$(3+4)5$	35
M_3	$2(2-1)$	2
M_4	$4(-2-5)$	-28
M_5	$(2-1)1$	1
M_6	$(3-2)(5+2)$	7
M_7	$(-1-4)(-2+1)$	5

Reconstruct $C_{11} = 36 - 28 - 1 + 5 = 12$, $C_{12} = 2 + 1 = 3$, $C_{21} = 35 - 28 = 7$, and $C_{22} = 36 - 35 + 2 + 7 = 10$. Thus

$$C = \begin{bmatrix} 12 & 3 \\ 7 & 10 \end{bmatrix},$$

agreeing with direct multiplication.

General block identity

Preserve left/right factor order:

$$\begin{aligned} C_{22} &= M_1 - M_2 + M_3 + M_6 \\ &= (ae + ah + de + dh) - (ce + de) + (af - ah) \\ &\quad + (ce + cf - ae - af) \\ &= cf + dh. \end{aligned}$$

Every canceled term occurs in identical factor order. The scalar example commutes, but induction uses matrix blocks $a, \ldots, h$, where generally $de \neq ed$. Swapping them invalidates cancellation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A02-E12 Maximum-subarray merging without scanning

Let (t, p, s, b) denote total, best nonempty prefix, suffix, and subarray sums. For adjacent left/right intervals L, R, use

$$\begin{aligned} t &= t_L + t_R, \\ p &= \max(p_L, t_L + p_R), \\ s &= \max(s_R, s_L + t_R), \\ b &= \max(b_L, b_R, s_L + p_R). \end{aligned}$$

A prefix is wholly left or contains all of the left interval plus a right prefix. Reverse the sides for suffixes. A best subarray is wholly left, wholly right, or crossing. These classifications are exhaustive. A singleton's four values are all A_i; induction on interval length proves all summaries. Carry endpoints alongside each selected summary when an interval is required.

Every internal merge takes constant time. With n leaves and $n - 1$ internal nodes,

$$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + \Theta(1) = \Theta(n).$$

Nonempty summaries also handle all-negative input. CROSS returns less information and rescans the halves at its parent. These are different algorithms, so their different costs are consistent.

A02-E13 Strengthening induction to pay lower-order costs

Because $5(n/2)^p = n^p$, direct substitution gives $cn^p + n^2$, which does not imply the same cn^p bound.

With $T(n) \le cn^p - dn^2$, substitution gives

$$5\left[c(n/2)^p - d(n/2)^2\right] + n^2 = cn^p - \frac{5d}{4}n^2 + n^2.$$

This is at most $cn^p - dn^2$ when $d \ge 4$. Choose $d = 4, c = 5$. The base gives $c - d = 1 = T(1)$, and substitution is actually equality, proving the exact expression

$$T(n) = 5n^p - 4n^2.$$

There are $5^{\lg n} = n^p$ leaves of cost 1, supplying $T(n) \ge n^p$. The expression gives $T(n) \le 5n^p$, so $T(n) = \Theta(n^{\lg 5})$. At $n = 2$, it gives $25 - 16 = 9 = 5T(1) + 4$.

Lower-order deductions at children provide enough slack to pay the parent's n^2 work.

A02-E14 Rounding and base cases

At $n = 2$, the child is $\lceil 10/6 \rceil = 2$, a self-call; sizes 3,4,5 also fail to shrink. Specifying only $T(1)$ is insufficient. Define fixed positive costs for $1 \le n \le 6$ instead. For $n \ge 7$, $\lceil 5n/6 \rceil \le n - 1$, guaranteeing termination.

Put $b = 6/5$ and $q_0 = n, q_{j+1} = \lceil q_j / b \rceil$ before stopping. If $|q_j - n/b^j| \le C$, next error is at most $C/b + 1$; $C = 6$ closes induction. Upward rounding also gives $q_j \ge n/b^j$.

For a direct upper bound, enlarge the base interval to $n \le 12$. At $j \ge \lceil \log_b(n/6) \rceil$, the upper estimate gives $q_j \le n/b^j + 6 \le 12$, so execution has stopped. While $n/b^j > 12$, the lower estimate prevents stopping. Thus depth is $\log_b n + O(1)$, and unit work per level gives $\Theta(\lg n)$ time.

With bases only through 6, reaching size at most 12 adds at most six further strictly decreasing calls. Both base intervals work; the original incomplete specification does not.

A02-E15 Regularity cannot be omitted

Always $f(n) \ge n^2$. With critical exponent $p = 1$, this provides the required polynomial gap with $\epsilon = 1$.

Along the infinite subsequence of even L, the child exponent is odd. Nonnegative work gives

$$T(n) \ge 2f(n/2) = 2(n/2)^3 = n^3/4.$$

Current root work is $f(n) = n^2$, so $T(n)/f(n) \ge n/4 \to \infty$. No fixed C bounds this ratio eventually; the recurrence is not even $O(f(n))$.

The regularity ratio itself is

$$\frac{2f(n/2)}{f(n)} = n/4,$$

which cannot be bounded by fixed $c < 1$. The counterexample fails case 3's regularity condition, not the theorem. Positive bases and nonnegative costs do not replace that hypothesis.

A02-E16 Finer boundaries than polynomial separation

Let $n = 2^L$. Stop at size 2, so internal depths are $j = 0, \ldots, L-2$. Renumber from the bottom by $r = L - j$, giving $r = 2, \ldots, L$. Level costs are respectively $n/(L-j)$ and $n/(L-j)^2$. Therefore

$$T_1(n) = \Theta(n) + n \sum_{r=2}^{L} \frac{1}{r} = \Theta(n \lg L),$$

$$T_2(n) = \Theta(n) + n \sum_{r=2}^{L} \frac{1}{r^2} = \Theta(n).$$

The first term in each expression pays for leaves. The harmonic sum is $\Theta(\lg L)$; the squared-reciprocal sum has fixed positive bounds. Restoring $L = \lg n$ gives $\Theta(n \lg \lg n)$ and $\Theta(n)$.

These are sufficiently-large-input asymptotics; small inputs retain their base costs. Neither is classical case 1: a logarithmic denominator cannot provide an n^ϵ gap for any fixed $\epsilon > 0$.

A02-E17 Parameter copying changes time

Keep original length N fixed and recurse on current length n. Only one child executes per level.

Arguments	Worst-case recurrence	Total from $n = N$
Indices or view	$V(n) = V(\lfloor n/2 \rfloor) + \Theta(1)$	$\Theta(\lg(N+1))$
Copy current interval	$C(n) = C(\lfloor n/2 \rfloor) + \Theta(n)$	$\Theta(N)$
Copy original array	$F(n; N) = F(\lfloor n/2 \rfloor; N) + \Theta(N)$	$\Theta(N \lg(N+1))$

The second row sums $N + N/2 + N/4 + \cdots = \Theta(N)$. The third always copies N records through $\Theta(\lg(N+1))$ worst-case levels. Replacing its $\Theta(N)$ toll by $\Theta(n)$ changes the model.

Assuming each copy occurs on entry, an immediate root match costs respectively $\Theta(1), \Theta(N), \Theta(N)$. A deepest successful search or a missing key attains the worst-case depth and corresponding table bounds.

A02-E18 Restore the original variable

Set $m = \lg n = 2^k$ and $S(m) = T(2^m)$. The base is $S(1) = T(2) = 1$, and

$$S(m) = 3S(m/2) + m.$$

The critical exponent is $p = \lg 3 > 1$. Linear local work is polynomially smaller than m^p, giving $S(m) = \Theta(m^p)$.

Alternatively, depth j costs $m(3/2)^j$ through k internal depths, and there are $3^k = m^p$ unit-cost leaves. The geometric sum has the same order as leaf work, establishing both bounds.

Restore $m = \lg n$:

$$T(n) = \Theta((\lg n)^{\lg 3}), \qquad n = 2^{2^k}.$$

The proposed $\Theta(n^{\lg 3})$ answer mistakes new variable m for original input n. This derivation covers the specified domain only, not a rounded recurrence on all integers.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A02-E19 Find the first duplicate

The main procedure first chooses $mid = 3$ and returns 3. That satisfies any-match search, not first-match search.

Search a half-open candidate interval $[lo, hi)$ for the first position with value at least x, allowing answer $N + 1$:

```
lo = 1; hi = N+1
while lo < hi:
    mid = lo + floor((hi-lo)/2)
    if A[mid] < x: lo = mid+1
    else: hi = mid
if lo <= N and A[lo] == x: return lo
return NIL
```

Before each probe, $1 \le lo \le hi \le N + 1$; every $i < lo$ has $A[i] < x$, and every legal $i \ge hi$ has $A[i] \ge x$. Both excluded regions are initially empty. Sortedness justifies each newly excluded region, preserving the invariant. The nonnegative integer $hi - lo$ strictly decreases. While $lo < hi$, $mid \le N$, so the loop never reads $A[N + 1]$.

The trace is $(lo, hi, mid) = (1, 7, 4) \to (1, 4, 2) \to (1, 2, 1)$, ending at $lo = hi = 2$. At termination, the invariant makes this the first position at least 2; the equality check makes it the first match.

Empty input starts at $(1, 1)$ and returns NIL. A key above all elements yields $N + 1$, guarded by short-circuit evaluation. Searching for 3 yields position 5, whose value 5 fails equality. Time is $O(\lg(N + 1))$ and auxiliary space $O(1)$.

A02-E20 Exact multiplication counts for powering

Exponents descend through 13,6,3,1:

Exponent	Computation	Returned value	Local multiplications
1	Base	2	0
3	$2^2 \cdot 2$	8	2
6	8^2	64	1
13	$64^2 \cdot 2$	8192	2

Right-shifting until only the leading 1 remains removes $\lfloor \lg n \rfloor$ bits, with one squaring per removed bit. Each removed 1 causes one extra multiplication by a. There are $\operatorname{popcount}(n) - 1$ such bits. This proves the formula, including zero multiplications at $n = 1$. Handle $n = 0$ separately by its base case.

At $n = 8$, reuse makes four calls along $8 \to 4 \to 2 \to 1$ and three multiplications. Duplication makes $1 + 2 + 4 + 8 = 15$ calls, with $1 + 2 + 4 = 7$ internal multiplications. Depth-first execution has four simultaneously live frames in either version. Total calls are not peak live frames.

A02-E21 Three costs in Fibonacci computation

The bases are $C_0 = C_1 = 1$. Put $D_n = C_n + 1$; then $D_n = D_{n-1} + D_{n-2}$ with $D_0 = D_1 = 2$. Induction gives $D_n = 2F_{n+1}$, hence $C_6 = 2F_7 - 1 = 25$.

Method	Arithmetic operations	Storage description
Naive recursion	$\Theta(\varphi^n)$	Depth $\Theta(n)$, with some returned values retained
Sequential addition	$\Theta(n)$ additions	Constant many integers, each at most $\Theta(n)$ bits
Matrix powering	$O(\lg(n+1))$ fixed-order matrix products	Recursive form has $O(\lg(n+1))$ frames; entries are large integers

Since $F_n = \Theta(\varphi^n)$, its binary length is $\Theta(n)$. Ordinary sequential addition at stage i processes $\Theta(i)$ bits, totaling $\Theta(\sum_{i=2}^{n} i) = \Theta(n^2)$ bit operations. The retained pair occupies $\Theta(n)$ bits. Matrix multiplication costs also grow with operand length; logarithmic arithmetic-operation count is not logarithmic bit time.

A02-E22 Leaf costs in a contracting tree

The root 16 produces 8,4,2; only 8 recurses, producing 4,2,1. Internal work is $16 + 8 = 24$, with five unit-cost leaves, so $T(16) = 29$. Counting $8 + 4 + 2 = 14$ as internal work and then charging leaves 4,2 again would double-count them.

Generally, internal work at depth j is at most $n(7/8)^j$, totaling at most $8n$. A ternary tree has $L = 2I + 1$; each internal node costs at least 8, so $I \le n$ and $L \le 2n + 1$. Adding leaves gives $O(n)$; root cost gives $\Omega(n)$.

For local work n^2, the child-work ratio is at most $(1/2)^2 + (1/4)^2 + (1/8)^2 = 21/64$, not $7/8$. For n^p, sum the pth powers of the child ratios. The linear-work shortcut does not apply unchanged.

A02-E23 Complete levels do not extend to the deepest leaf

List only nodes reached at each depth; stopped nodes have no descendants:

Depth	Node sizes	Internal work	Leaf cost
0	18	18	0
1	6,12	18	0
2	2,4,4,8	16	1
3	1,3,1,3,2,6	12	3
4	1,2,1,2,2,4	4	5
5	1,3	3	1
6	1,2	0	2

Internal work totals $18 + 18 + 16 + 12 + 4 + 3 = 71$; the 12 leaves cost 12. Thus $T(18) = 83$.

Leaves appear at depth 2, reducing active mass below 18. Filling each later level to 18 adds nonexistent work, hence constructs an upper bound. A lower bound retains only the complete levels before any path terminates, then proves there are $\Omega(\lg n)$ such levels.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A02-E24 A recurrence inequality need not give a tight bound

The constant function $T_a(n) = 1$ satisfies $1 \leq 2 + n$. The function $T_b(n) = n(1 + \lg n)$ also satisfies the condition, with equality. One is constant and the other is $\Theta(n \lg n)$, so the given inequality does not imply a tight bound.

Define $U(1) = 1$ and $U(n) = 2U(n/2) + n$. Strong induction proves $T(n) \leq U(n)$ by substituting the smaller-input inequalities. Expansion gives $U(n) = n(1 + \lg n)$, proving an $O(n \lg n)$ upper bound for every admissible T.

A matching lower bound follows if both recursive calls really execute, every internal call performs at least cn local work for fixed $c > 0$, and base costs are bounded below positively. Then complete levels supply $\Omega(n \lg n)$. Nonnegativity alone supplies no such work guarantee.

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study